

Freshness Ads

SDK quảng cáo AdMob cho Android — waterfall theo tier, native pool, consent UMP, cấu hình bằng Remote Config.

Phiên bản 1.1.0 · Google Mobile Ads Next-Gen SDK 1.2.1 · minSdk 26 · Hướng dẫn sử dụng

Nội dung

1. SDK này giải quyết chuyện gì
2. Cài đặt
3. File cấu hình `ads_id_config.json`
4. Khởi tạo trong Application
5. Màn chờ trước khi hiện quảng cáo
6. Consent (UMP) và luồng splash
7. Interstitial
8. Native — một slot
9. Native — pool cho danh sách
10. Native full màn
11. Banner
12. Rewarded
13. App-open
14. Remote Config
15. Sự kiện và doanh thu quảng cáo
16. Jetpack Compose
17. ProGuard / R8
18. Mediation Unity Ads
19. Test và debug
20. Những lỗi làm tụt show rate
21. Thay đổi theo phiên bản

1. SDK này giải quyết chuyện gì

Gọi thẳng Google Mobile Ads SDK thì mỗi màn hình phải tự lo: load, cache, huỷ ad đúng lúc, chờ consent xong mới request, xử lý no-fill, đổi ad unit id khi cần. Làm lại từng đó ở mỗi app là chỗ sinh bug và sinh lãng phí request.

SDK này gom lại thành bốn thứ:

Thứ	Nghĩa là gì
Catalog	Ad unit id không nằm trong code. Chúng nằm trong <code>assets/ads_id_config.json</code> , và Remote Config ghi đề được từng placement — đối id, tắt một chỗ đặt ad, siết thời gian chờ, tất cả không cần release.
Waterfall	Mỗi placement khai được nhiều id xếp theo thứ tự. Id đầu không fill thì tự xuống id kế. Dừng để đặt tầng high-floor phía trên.
Ngân sách thời gian	Mỗi format có deadline riêng, và deadline đó chia cho từng tier. Hết ngân sách thì dừng, không để user ngồi chờ một request không bao giờ về.
Vòng đời	Ad được cache, được huỷ đúng lúc view chết, và fill về muộn vẫn được giữ cho lần load sau thay vì vứt đi.

Một đầu vào duy nhất: `AdsGraph.adsManager`. Không có Hilt/Dagger, không cần kapt/KSP.

2. Cài đặt

2.1 Thêm repository và dependency

Trong `settings.gradle.kts`:

```
dependencyResolutionManagement {
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") }
    }
}
```

Trong `app/build.gradle.kts`:

```
dependencies {
    implementation("com.github.vuhuyvqh2112:freshness-ads:1.1.0")
}
```

App dùng Jetpack Compose thêm module wrapper (kéo theo SDK chính, không phải khai cả hai) **1.1.0+**:

```
implementation("com.github.vuhuyvqh2112:freshness-ads-compose:1.1.0")
```

Không cần khai thêm GMA SDK, UMP, Unity adapter hay Firebase Config — POM của SDK đã mang theo hết. Từ 1.0.1, `androidx.lifecycle` cũng đi kèm (khai `api`) vì `bindNativeAdFromCache` là extension trên `LifecycleOwner`.

Phần khung `androidx` còn lại (`appcompat`, `material`, `coroutines`) SDK khai bằng `implementation`: chúng vẫn nằm trong runtime classpath của app nhưng không lộ ra API, nên app muốn gọi trực tiếp thì tự khai như bình thường.

2.2 Firebase **tùy chọn từ 1.1.0**

Có Firebase thì id và setting ghi đề được bằng Remote Config (mục 14); không có thì SDK chạy bằng `assets/ads_id_config.json` và ghi một dòng cảnh báo lúc khởi động. Muốn dùng Remote Config:

```
// build.gradle.kts (root) – lấy version mới nhất trên Google Maven
plugins {
    id("com.google.gms.google-services") apply false
    id("com.google.firebase.crashlytics") apply false // tùy chọn
}

// app/build.gradle.kts
plugins {
    id("com.google.gms.google-services")
    id("com.google.firebase.crashlytics")
}
```

Đặt `google-services.json` vào thư mục `app/`.

Crashlytics không còn bắt buộc.

Từ 1.1.0 SDK khai `firebase-crashlytics` bằng `compileOnly`: app có Crashlytics thì crash nội bộ của UMP được ghi non-fatal, không có thì chỉ còn dòng log. Trước 1.1.0 thiếu plugin Crashlytics là app crash ngay lúc khởi động (The Crashlytics build ID is missing) — nâng lên 1.1.0 là hết.

2.3 AdMob app id trong manifest

```
<application>
    <meta-data
        android:name="com.google.android.gms.ads.APPLICATION_ID"
        android:value="ca-app-pub-XXXXXXXXXXXXXXXX~YYYYYYYYYY" />
</application>
```

Đây là APP ID, không phải ad unit id.

App id có dấu `~`, ad unit id có dấu `/`. Điền nhầm là app crash ngay lúc khởi động, không phải "không có ad".

Quyền `INTERNET`, `AD_ID` và `install-referrer` đã nằm trong manifest của SDK, không phải khai lại.

3. File cấu hình `ads_id_config.json`

Chép `samples/ads_id_config.json` vào `app/src/main/assets/ads_id_config.json`. Đây là bản mặc định đóng gói trong APK, dùng khi Remote Config chưa fetch về kịp (lần mở app đầu tiên, hoặc mất mạng).

```
{
  "enableAllAds": true,
  "settings": {
    "splashTimeoutMs": 30000,
    "interMinIntervalMs": 20000,
    "rewardTimeoutMs": 20000,
    "free_episodes_count": 3
  },
  "placements": {
    "inter_splash": {
      "format": "interstitial",
      "enable": true,
      "hf": false,
      "ids": [
        "ca-app-pub-xxx/high-floor",
        { "id": "ca-app-pub-xxx/mid", "enable": false },
        "ca-app-pub-xxx/all-price"
      ]
    },
    "native_home": {
      "format": "native",
      "enable": true,
      "baseMs": 30000,
      "tierCapMs": 8000,
      "ceilingMs": 120000,
      "ids": [
        { "id": "ca-app-pub-xxx/native-home", "enable": true }
      ]
    }
  }
}
```

Trường	Ý nghĩa
<code>enableAllAds</code>	Công tắc tổng. <code>false</code> là tắt sạch mọi format.
<code>placements.<key></code>	Tên chỗ đặt ad. Do bạn tự đặt — SDK không ép danh sách nào.
<code>format</code>	<code>banner</code> · <code>native</code> · <code>interstitial</code> · <code>rewarded</code> · <code>rewardedInter</code> 1.1.0+ · <code>appOpen</code> . Quyết định test id dùng ở debug build.
<code>enable (placement)</code>	<code>false</code> = tắt cả chỗ đặt ad này.
<code>ids</code>	Thứ tự waterfall. Id đầu thử trước; fail mới xuống id kế. Quy ước: id cuối là all-price, mọi id trước nó là high-floor. Mỗi phần tử viết được dạng chuỗi <code>"ca-app-pub..."</code> hoặc object <code>{ "id", "enable" }</code> 1.1.0+ .
<code>enable (id)</code>	<code>false</code> = tắt riêng một tầng, giữ nguyên các tầng khác.
<code>hf</code> 1.1.0+	<code>false</code> = tắt cả tầng high-floor, chỉ chạy id cuối. Thiếu hoặc <code>true</code> = dùng hết <code>ids</code> . Một công tắc để ops bật HF khi có inventory, không phải sửa từng id.
<code>settings.<key></code> 1.1.0+	Key tùy ý của app (số, boolean, chuỗi). Đọc bằng <code>ads.remoteConfig.getLong("key", default) / bool / string</code> — xem mục 14.
<code>baseMs / tierCapMs / ceilingMs</code>	Ghi đè ngân sách thời gian mặc định của format. Bỏ trống là dùng mặc định.

Placement không tồn tại = tắt.

Key không có ở cả Remote Config lẫn file assets sẽ resolve ra rỗng, và mọi lệnh load cho nó im lặng bỏ qua. Không crash, nhưng cũng không có ad — gõ sai tên placement là lỗi khó thấy nhất khi tích hợp.

Hai placement SDK tự gọi

Hầu hết placement được truyền vào từ call site. Riêng hai cái này SDK tự gọi nên phải khai tên trong `AdsConfig` (mặc định `open_all` và `inter_splash`):

Placement	Vì sao SDK phải biết sẵn
<code>open_all</code>	App-open ad load ngầm theo vòng đời process — không có màn hình nào truyền key vào.
<code>inter_splash</code>	Interstitial của splash có ngân sách thời gian riêng (không giãn theo số tier), nên SDK phải nhận ra nó giữa các interstitial khác.

4. Khởi tạo trong Application

```
class MyApp : Application() {

    override fun onCreate() {
        super.onCreate()

        AdsGraph.install(
            application = this,
            config = AdsConfig(
                openAdPlacement = "open_all",
                splashInterstitialPlacement = "inter_splash",
                // key placement -> số ad giữ sẵn trong pool
                nativePools = mapOf("native_home" to 1),
                // SDK tự vẽ màn chờ trước khi bung ad toàn màn (xem mục 5)
                showDefaultLoadingUi = true,
            ),
        )

        // App-open ad phải được che bằng một Activity đực trước khi hiện
        // (yêu cầu policy của AdMob: user không được thấy nội dung app sau lưng ad).
        AdsGraph.adsManager.setOnShowOpenAdListener {
            AdsGraph.adsManager.topActivity.get()?.let { OpenAdActivity.start(it) }
        }
    }
}
```

`install` gọi một lần; lần thứ hai là no-op. Sau đó dùng `AdsGraph.adsManager` ở mọi nơi.

Nối với IAP

App có mua-gói-bỏ-quảng-cáo thì truyền một lambda; SDK đọc lại mỗi lần kiểm tra nên mua xong là tắt ads ngay, không cần gọi gì thêm:

```
AdsConfig(
    /* ... */
    isPremium = { billing.hasRemoveAds },
)
```

Cần logic phức tạp hơn (thêm cờ riêng của app, A/B test...) thì truyền hẳn `AdsDataStore` vào `install`; khi đó `isPremium` không được đọc:

```
AdsGraph.install(
    application = this,
    config = AdsConfig(/* ... */),
    dataStore = object : AdsDataStore {
        override var isPurchased: Boolean = billing.hasRemoveAds
        override val isAdEnabled: Boolean
            get() = !isPurchased && remoteFlags.adsOn
    },
)
```

5. Màn chờ trước khi hiện quảng cáo 1.0.3+

SDK tự vẽ một màn chờ che toàn màn (nền trắng, spinner xanh) trong lúc nạp interstitial và rewarded, rồi tắt ngay khi quảng cáo chiếm màn hình. App không phải làm gì.

Điểm quan trọng: **màn chờ được giữ tối thiểu 500ms trước khi quảng cáo bung ra, kể cả khi ad đã preload sẵn.** Đây không phải để câu giờ — nó là khuyến nghị của chính sách AdMob:

Recommended interstitial implementations — Google AdMob Help

“it is recommended that a delay is inserted after the end of a level and before the display of an interstitial ad. This delay could take the form of a *loading* or *please wait* screen, or a progress-bar/wheel.”

Mục đích là cho ngón tay đang bấm kịp dừng lại trước khi quảng cáo chiếm màn hình, giảm click nhầm. Google không nêu con số cụ thể; 500ms là lựa chọn của SDK.

Đo trên máy thật với ad đã nằm sẵn trong cache: không có khoảng chờ này thì từ lúc bấm tới lúc quảng cáo hiện chỉ **15ms** — người dùng không kịp thấy gì.

Tùy chỉnh

Muốn gì	Làm sao
Đổi thời gian chờ tối thiểu	<code>InterAdConfig(minLoadingMs = 800L) / RewardAdConfig(minLoadingMs = 800L) . 0 =</code> bung ngay khi có ad, nhưng màn chờ vẫn loé một frame — muốn không thấy gì thì dùng <code>isShowLoading = false</code> ở dòng dưới.
Đổi màu	Khai lại đúng tên trong <code>res/values/colors.xml</code> của app: <code>ads_loading_background</code> , <code>ads_loading_spinner</code> .
Tự vẽ giao diện riêng	<code>AdsConfig(showDefaultLoadingUi = false)</code> rồi observe <code>AdsGraph.adLoading.isLoading</code> .
Tắt màn chờ ở một chỗ	<code>isShowLoading = false</code> trong config của format đó. Tắt là tắt cả khoảng chờ tối thiểu : màn hình đó phải tự giữ khoảng chờ bằng loading riêng của mình.

```
<!-- Ghi đè màu, đặt trong res/values/colors.xml của app -->
<color name="ads_loading_background">#FFFFFF</color>
<color name="ads_loading_spinner">#FF1A73E8</color>
```

Tự vẽ thì đừng bỏ khoảng chờ.

Tắt `showDefaultLoadingUi` để dùng giao diện riêng là hợp lý. Nhưng bỏ hẳn khoảng chờ giữa cú chạm và lúc quảng cáo bung ra thì đi ngược khuyến nghị ở trên, và đó đúng là tình huống sinh click nhầm.

6. Consent (UMP) và luồng splash

`launchSplashAdFlow` làm trọn gói: gom consent (có timeout), rồi hiện interstitial splash nếu placement đó đang bật.

Splash: tắt màn chờ của SDK.

Splash đã là màn chờ rồi. Để `isShowLoading` mặc định là màn chờ trắng của SDK phủ lên splash của app suốt lúc chờ fill (tối 30 giây). Đặt `isShowLoading = false` trong config của splash như dưới đây.

```
class SplashActivity : AppCompatActivity() {

    private val ads get() = AdsGraph.adsManager

    private val splashInterConfig = InterAdConfig(
        placement = "inter_splash",
        reload = false,
        timeOut = 30_000L,
        // Splash tự là màn chờ, không để SDK phủ thêm một lớp trắng lên nó
        isShowLoading = false,
        // Một lần một unit, không retry: waterfall tuôn tự nhanh hơn. Retry làm id
        // chính fill muộn, tới lúc đó user đã rời splash – matched nhưng không show.
        retryCount = 1,
    )

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Bắn request NGAY, song song với consent và Remote Config.
        if (ads.isAdEnabled("inter_splash")) ads.loadInterAd(splashInterConfig)

        ads.setIsSplash(true)
        lifecycleScope.launch {
            val shown = try {
                ads.launchSplashAdFlow(splashInterConfig)
            } finally {
                // finally, KHÔNG phải dòng sau lệnh suspend: user bấm back giữa lúc chờ
                // là scope bị huỷ và chờ splash kết true, chặn mọi app-open ad về sau.
                ads.setIsSplash(false)
            }
            goToNextScreen()
        }
    }
}
```

Bắt buộc cho user EEA/UK.

Policy của Google yêu cầu app có lối vào để đổi/rút lại consent. Kiểm tra `ads.isPrivacyOptionsRequired` rồi hiện một mục trong Settings gọi `ads.showPrivacyOptionsForm(activity)`. Thiếu cái này là rủi ro bị gỡ app.

Chờ SDK khởi tạo xong

Mọi lệnh load đều chờ `MobileAds.initialize` hoàn tất trước khi chạm SDK. Ngưỡng chờ khác nhau theo bản chất từng format:

Format	Chờ tối đa	Vì sao
Banner, Native	60 giây	Bị động, không ai đang chờ. Hiện muộn vẫn hơn không hiện — và không có gì kích hoạt lại lệnh load khi user vẫn ngồi trên màn hình.
Interstitial, Rewarded	15 giây	Có người dùng đang chờ sau spinner. Thà bỏ quảng cáo còn hơn giữ họ lâu hơn. Đây là ngưỡng chờ <i>init</i> cố định; thời gian spinner thật sự hiện do <code>timeout</code> của config quyết định (inter mặc định 6 giây), và tăng <code>timeout</code> không kéo dài được ngưỡng init này.
App-open	15 giây	Hiện muộn 30 giây sau khi user đã vào app thì vô nghĩa.

Thấy `SKIP ... (SDK not ready)` trong logcat ngay trước dòng `MobileAds (next-gen) initialized` nghĩa là init chạy quá lâu — xem mục 18, gần như luôn là do thiếu mediation group.

7. Interstitial

```
private val interConfig = InterAdConfig(
    placement = "inter_next_screen",
    reload = true,           // tự load lại sau khi hiện xong
    retryCount = 2,
    timeOut = 6_000L,
    isShowLoading = true,   // màn chờ trong lúc nạp (mục 5)
    minLoadingMs = 500L,    // giữ màn chờ tối thiểu, kể cả khi ad đã có sẵn
)

// Warm sớm, ở màn trước đó
ads.loadInterAd(interConfig)

// Lúc chuyển màn
lifecycleScope.launch {
    ads.showInterAd(interConfig, placement = "inter_next_screen")
    startActivity(Intent(this@Screen, NextActivity::class.java))
}
```

Nạp và hiện trong một lệnh 1.0.3+

Chỗ đặt ad không preload trước được (user bấm rồi mới biết cần quảng cáo) thì dùng

`loadAndShowInterAd` :

```
lifecycleScope.launch {
    ads.loadAndShowInterAd(interConfig, placement = "inter_next_screen")
    startActivity(Intent(this@Screen, NextActivity::class.java))
}
```

Nó nạp nếu chưa có sẵn rồi hiện, màn chờ che khoảng nạp. Vẫn tôn trọng frequency cap và công tắc placement như `showInterAd`. Ad đã nằm trong cache thì không phát thêm request nào.

`showInterAd` và `loadAndShowInterAd` đều trả về sau khi user đóng ad, và trả `false` nếu không có gì để hiện. Luôn đi tiếp bất kể kết quả — đừng chặn điều hướng vì một quảng cáo không fill.

SDK tự áp **frequency cap** giữa hai interstitial bất kỳ (mặc định 20s, chỉnh bằng `settings.interMinIntervalMs`). Bấm nhanh qua nhiều màn sẽ không ăn liên tiếp hai quảng cáo toàn màn.

Vì sao không hiện: `AdShowOutcome` 1.1.0+

Bản `...Outcome` của mọi lệnh show trả lời lý do thay vì một `false` câm — để log, A/B, hoặc quyết định có chờ thêm không:

```
when (val outcome = ads.showInterAdOutcome(interConfig)) {
    AdShowOutcome.SHOWN -> analytics.log("inter_shown")
    AdShowOutcome.FREQUENCY_CAP,
    AdShowOutcome.PLACEMENT_OFF -> Unit           // bình thường, đi tiếp
    AdShowOutcome.NO_FILL -> analytics.log("inter_no_fill")
    else -> Timber.w("inter: $outcome")
}
```

Giá trị	Nghĩa
SHOWN	Đã hiện và người dùng đã đóng.
DISABLED	Premium hoặc <code>enableAllAds = false</code> .
PLACEMENT_OFF	Placement tắt hoặc không có id trong <code>ads_id_config</code> .
CONSENT_MISSING	Người dùng EEA chưa đồng ý consent.
FREQUENCY_CAP	Chưa đủ khoảng cách tối thiểu giữa hai interstitial.
ALREADY_SHOWING	Đang có ad toàn màn khác, hoặc một lần show cùng placement đang chạy.
NO_FILL	Hết <code>timeOut</code> mà không có ad.
NO_ACTIVITY / SHOW_FAILED	Không còn activity, hoặc <code>show()</code> hỏng (ad hết hạn, activity đang finish).

Ad đã nạp có hạn dùng 1 giờ **1.1.0+**.

AdMob coi interstitial/rewarded là hết hạn sau khoảng một giờ; đem ad cũ đi show là `SHOW_FAILED` và người dùng nhìn spinner vô ích. SDK tự bỏ ad quá hạn và nạp lại — preload ở màn đầu rồi để user đi lâu vẫn an toàn.

8. Native — một slot

Đặt view vào layout. Chọn template theo chỗ đặt:

```
<com.freshness.ads.natives.LoadingNativeAdView1
    android:id="@+id/nativeAd"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Class	Dùng cho	Chiều cao
<code>LoadingNativeAdView1 ... View4</code>	Bố cục chuẩn, khác nhau ở vị trí media và nút CTA	<code>wrap_content</code>
<code>LoadingNativeAdSmallView</code>	Dài nhỏ, không có ảnh lớn	<code>wrap_content</code>
<code>LoadingNativeAdHomeView</code>	Khối lớn trên màn chính, ảnh phủ kín card	khai cụ thể
<code>LoadingNativeAdHomeCompactView</code>	Bản gọn của trên, ảnh vuông bên trái	<code>wrap_content</code>
<code>LoadingNativeAdListView / CategoryView</code>	Item chèn giữa danh sách	<code>wrap_content</code>
<code>LoadingNativeCtaTopView / CtaBottomView</code>	CTA nằm trên / dưới	<code>wrap_content</code>
<code>LoadingNativeAdDisplayView</code>	Khối lớn, ảnh phủ kín card	khai cụ thể
<code>LoadingNativeFullscreenView</code>	Native chiếm trọn màn	<code>match_parent</code>

Tiền tố `Loading` nghĩa là view đã kèm shimmer: nó tự hiện khung xám trong lúc chờ và tự đổi sang ad khi có.

Hai template full-bleed cần chiều cao cụ thể.

HomeView và DisplayView để MediaView phủ kín card, nên chiều cao hoàn toàn do container quyết định. Cho wrap_content thì card co lại vừa đủ phần chữ và vùng ảnh co theo — đo được 70dp, trong khi Google yêu cầu native có video phải đạt tối thiểu **120×120dp**.

Từ 1.0.5 SDK giữ sàn 120dp nên không còn vi phạm, nhưng vẫn nên khai chiều cao cụ thể (ví dụ 260dp) để kiểm soát bố cục thay vì dựa vào sàn.

Trong Activity/Fragment:

```
private fun showNativeAd() {
    if (!ads.isAdEnabled("native_detail")) {
        binding.nativeAd.isVisible = false
        return
    }
    val service = ads.nativeAdService
    service.preloadIfEmpty(applicationContext, key = "native_detail", placement =
"native_detail")
    bindNativeAdFromCache(
        container = binding.nativeAd,
        nativeAdService = service,
        cacheKey = "native_detail",
        // true cho màn một-lần: ad được huỷ khi màn chết, NHƯNG chỉ khi nó đã thật
        // sự ăn impression. Fill chưa kịp hiện thì giữ lại cho lần vào sau.
        releaseOnDestroy = true,
    )
}
```

Hai khái niệm khác nhau: key và placement .

placement quyết định load id nào (key trong ads_id_config).

key định danh chỗ hiển thị trong cache. Hai màn dùng chung một placement vẫn phải có key riêng, nếu không màn thứ hai sẽ bind đúng cái ad màn thứ nhất đang giữ.

Tuỳ chọn native: tiếng video, tỉ lệ media, AdChoices 1.1.0+

Mặc định toàn cục đặt trong AdsConfig(nativeAdOptions = NativeAdOptions(...)) , từng placement ghi đè trong ads_id_config :

```
// AdsConfig – mặc định cho mọi placement
nativeAdOptions = NativeAdOptions(
    videoMuted = true, // mặc định: video native KHÔNG tự phát tiếng
    mediaAspectRatio = NativeMediaAspectRatio.ANY,
    adChoicesPlacement = AdChoicesCorner.TOP_RIGHT,
)

// ads_id_config – riêng một placement
"native_home": { "mediaAspectRatio": "landscape", "adChoicesPlacement": "bottom_left" }
```

Chọn mediaAspectRatio khớp với template đang dùng (HomeView / DisplayView là landscape) để Google trả creative đúng tỉ lệ, ảnh không bị cắt.

Nạp lại slot sau click và sau video 1.1.0+

Với slot đơn (`bindNativeAdFromCache`), SDK tự thay ad mới vào cùng chỗ khi người dùng bấm vào ad rồi quay lại app, và khi video của native chạy hết. Ad cũ đã tạo click / đã xem xong; ad mới là thêm một impression cho cùng chỗ đặt, không tốn thêm UI. Slot giữ nguyên ad cũ cho tới khi ad mới về (không nháy shimmer), no-fill thì giữ ad cũ.

Tắt bằng `AdsConfig(nativeRefill = false)` hoặc Remote Config `settings.nativeRefill = false`. Pool cho danh sách (mục 9) không áp: pool đã tự nạp theo impression.

Template từ layout XML của app 1.1.0+

Không cần subclass hai class cho mỗi thiết kế nữa. Vẽ layout với root là `NativeAdView` id `native_ad_view` và các id quy ước — đây cũng là id 13 template có sẵn của SDK đang dùng, nên copy XML của SDK ra sửa là chạy:

Id	View	
<code>native_ad_view</code>	<code>NativeAdView</code> (root)	bắt buộc
<code>media_view</code>	<code>MediaView</code>	tùy chọn — thiếu id nào thì asset đó không hiển thị
<code>icon</code>	<code>ImageView</code>	
<code>primary</code> / <code>body</code> / <code>cta</code> / <code>advertiser</code> / <code>price</code>	<code>TextView</code>	
<code>rating_bar</code>	<code>AppCompatRatingBar</code>	
<code>rate_price_container</code>	View bọc rating + price, ẩn khi cả hai trống	

```
<!-- XML: shimmer mặc định của SDK, hoặc app:adShimmerLayout riêng -->
<com.freshness.ads.natives.LoadingNativeAdTemplateView
    android:id="@+id/nativeAd"
    android:layout_width="match_parent"
    android:layout_height="240dp"
    app:adLayout="@layout/my_native_card" />

// Code
val view = LoadingNativeAdTemplateView(context, R.layout.my_native_card)

// Bind y hệt các template có sẵn
bindNativeAdFromCache(binding.nativeAd, ads.nativeAdService, "native_detail", releaseOnDestroy = true)
```

Preload sớm một màn

`preloadIfEmpty` là idempotent: gọi lại khi đã có ad hoặc đang load thì không phát thêm request. Nhờ vậy gọi được từ màn trước để ad sẵn sàng trước khi user tới:

```
// Ở màn A, hàm sẵn native của màn B
ads.nativeAdService.preloadIfEmpty(applicationContext, "native_b", "native_b")
```

9. Native — pool cho danh sách

Danh sách chèn nhiều vị trí ad thì dùng `AdPoolManager` : nó preload sẵn một số ad và phát ngay khi adapter cần, thay vì load trong `onBindViewHolder` (vừa giật vừa phát dư request).

```
// Khai pool lúc install
AdsGraph.install(
    application = this,
    config = AdsConfig(nativePools = mapOf("native_feed" to 3)),
)

// Hàm khi mở màn
val pool = ads.adPoolManager
pool.preloadIfEmpty("native_feed")

// Trong adapter
val ad = pool.getAd("native_feed", key = "native_feed_$position", owner = holder)
if (ad != null) {
    holder.adView.setNativeAd(ad)
} else if (pool.isPlacementExhausted("native_feed")) {
    holder.adView.isVisible = false // hết ad thật, thu gọn slot
}                                     // còn lại: để shimmer chạy tiếp

// Khi holder bị recycle
pool.releaseAd("native_feed", key = "native_feed_$position", owner = holder)
```

Muốn bind lại ngay khi có ad mới về, lắng nghe `pool.adReadyEvents` :

```
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        pool.adReadyEvents.collect { placement ->
            if (placement == "native_feed") adapter.notifyAdSlotsChanged()
        }
    }
}
```

Pool size phải bằng số slot hiển thị CÙNG LÚC.

Màn chỉ có một slot tĩnh mà khai 2 thì ad thứ hai không có đường ra: slot đã có ad rồi thì lần bind sau bỏ qua. Kết quả là mỗi phiên phát dư một request không bao giờ đổi được impression — show rate tụt thẳng. Chỉ tăng pool khi có feed thật sự hiển thị nhiều ad song song.

`owner` là tham số quan trọng: RecyclerView có thể tạo holder mới cho cùng một vị trí *trước khi* holder cũ bị recycle (ItemAnimator còn dựng hân holder thứ hai để crossfade). Truyền `owner` để pool chỉ huỷ ad khi không còn holder nào giữ nó. Không truyền thì holder cũ sẽ huỷ mất ad mà holder mới đang hiển thị, và frame sau crash *"trying to use a recycled bitmap"*.

10. Native full màn

Native chiếm trọn màn hình dùng làm fallback khi interstitial không fill, hoặc chèn giữa các bước onboarding. Dựng bằng một Activity riêng:

```

class NativeFullscreenActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityNativeFullBinding.inflate(layoutInflater)
        setContentView(binding.root)

        val service = AdsGraph.adsManager.nativeAdService
        // Fill có thể'biến mất giữa lúc màn gọi kiểm tra và lúc màn này dựng xong.
        if (!service.isReady("native_full")) { finish(); return }

        bindNativeAdFromCache(
            container = binding.nativeFull,
            nativeAdService = service,
            cacheKey = "native_full",
            releaseOnDestroy = true,
        )
        binding.btnClose.setOnClickListener { finish() }
    }
}

```

Luôn phải có đường thoát.

Nút đóng phải hiện, và BACK phải chạy được. Muốn giữ impression đủ lâu để tính thì hoãn nút đóng vài giây bằng một `CountDownTimer` — nhưng đừng chặn BACK, và đừng để màn hình nào không thoát được.

11. Banner

```

<FrameLayout
    android:id="@+id/bannerContainer"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

```

```

ads.bannerAdService.loadBannerAds(
    context = this,
    bannerView = binding.bannerContainer,
    placement = "banner_home",
    size = BannerSize.ANCHORED,    //mặc định; INLINE cho feed cuộn, MREC 300×250
    onLoadSuccess = { binding.bannerContainer.isVisible = true },
    onLoadFail = { binding.bannerContainer.isVisible = false },
)

```

Container banner phải `wrap_content` .

Từ 1.0.1 banner dùng *large anchored adaptive* — bản Google thay cho API cũ đã deprecated. Nó **cao hơn** anchored thường. Container nào khoá `layout_height` cứng sẽ cắt mất quảng cáo.

BannerSize 1.1.0+	Dùng khi
ANCHORED	Neo đỉnh/đáy màn. Bản <i>large</i> anchored adaptive, hành vi từ 1.0.1.
INLINE	Nằm trong nội dung cuộn (feed, giữa bài). Google trả chiều cao theo creative nên cao hơn anchored — container vẫn <code>wrap_content</code> .
MREC	300×250dp cố định, cho chỗ đặt dạng thẻ.

Tham số `isCollapsible` ở bản cũ **chưa bao giờ có tác dụng**: GMA next-gen 1.2.1 không có API collapsible banner. Từ 1.1.0 nó bị đánh dấu deprecated, call site cũ vẫn compile và vẫn nạp anchored banner thường.

12. Rewarded

```
private val rewardConfig = RewardAdConfig(
    placement = "reward_unlock",
    retryCount = 2,
    timeOut = 20_000L,
    isShowLoading = true,
    minLoadingMs = 500L,
)

lifecycleScope.launch {
    var earned = false
    val shown = ads.loadAndShowRewardAd(
        config = rewardConfig,
        placement = "reward_unlock",
        onShow = { /* quảng cáo đã hiện toàn màn */ },
        onReward = { earned = true },
    )
    if (shown && earned) unlockFeature()
    else showCouldNotLoadMessage()
}
```

`loadAndShowRewardAd` **1.0.3+** hợp với rewarded hơn cả: người dùng bấm “xem quảng cáo nhận thưởng” xong mới cần tới ad, nên preload trước thưởng chỉ tạo request không đổi được impression. Vẫn có `loadRewardAd` + `showRewardAd` nếu muốn tách hai bước.

onShow và onReward chạy trên main thread.

1.0.3+ GMA Next-Gen bắn hai callback này trên luồng nền của nó. Trước 1.0.3, SDK chuyển thẳng cho app nên mọi thao tác UI trong đó đều crash (`Can't toast on a thread that has not called Looper.prepare()`). Nay SDK đưa về main trước khi giao cho app — cộng thưởng, cập nhật UI, ghi storage đều làm thẳng trong callback được.

Đừng chờ giá trị trả về để tắt spinner của bạn.

Hàm chỉ trả về *sau khi* user đóng quảng cáo. Call site nào đang giữ spinner riêng mà đợi giá trị trả về thì spinner nằm dưới quảng cáo suốt cả video rồi loé lên một nhíp lúc đóng. Dùng `onShow`. (Màn chờ mặc định của SDK ở mục 5 tự tắt đúng lúc, không cần xử lý.)

Và: hỏi quyền / kiểm điều kiện **trước** khi mở quảng cáo. Bắt user xem hết video rồi mới báo “thiếu quyền” là lừa họ bỏ công lấy một thứ không nhận được.

Rewarded interstitial **1.1.0+**

Format “xem để nhận thưởng” nhưng hiện ở điểm chuyển màn tự nhiên, không cần người dùng bấm. Cùng `RewardAdConfig` và cùng hành vi với rewarded (nạp-rồi-hiện, màn chờ, hạn dùng, callback), chỉ khác placement khai `"format": "rewardedInter"`:

```
lifecycleScope.launch {  
    ads.loadAndShowRewardedInterAd("reward_inter_level_end", onReward = { grantBonus() })  
    goToNextLevel()  
}  
// Preload / bản Outcome: loadRewardedInterAd, showRewardedInterAdOutcome,  
loadAndShowRewardedInterAdOutcome
```

13. App-open

App-open ad hiện khi user quay lại app từ background. SDK tự theo dõi vòng đời process; app chỉ cần cung cấp một Activity để che nội dung:

```
// Application.onCreate
AdsGraph.adsManager.setOnShowOpenAdListener {
    AdsGraph.adsManager.topActivity.get()?.let { OpenAdActivity.start(it) }
}

// OpenAdActivity: theme trong suốt/đục, không layout
class OpenAdActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        onBackPressedDispatcher.addCallback(this) { /* chặn back khi ad đang chạy */ }
        lifecycleScope.launch {

            AdsGraph.adsManager.openAdService.showAdIfAvailable(WeakReference(this@OpenAdActivity))
                finish() // đóng dù có ad hay không, đừng để lại màn trắng
                overridePendingTransition(0, 0)
        }
    }
}
```

Tạm tắt app-open cho một lần quay lại (ví dụ user vừa đi ra ngoài chọn ảnh rồi quay vào):

```
ads.setSkipOpenAd(true)
```

Màn nào **không bao giờ** muốn app-open đè lên (thanh toán, chọn file, activity quảng cáo của SDK khác) thì khai một lần thay vì nhớ gọi `setSkipOpenAd` mỗi chỗ **1.1.0+**:

```
// AdsConfig
openAdExcludedActivities = setOf(BillingActivity::class.java, FilePickerActivity::class.java)

// hoặc lúc chạy
ads.openAdService.excludeActivity(SomeActivity::class.java)
```

Cần pause video/nhạc trong lúc bất kỳ ad toàn màn nào đang hiện thì observe một cờ gộp thay vì ba:

`ads.isFullScreenAdShowing` **1.1.0+**. `Activity.onPause` không đủ vì màn kế tiếp có thể được dựng ngay dưới quảng cáo.

Bắt buộc: che nội dung app.

Policy AdMob không cho user thấy nội dung app phía sau app-open ad. Không có Activity che là vi phạm.

14. Remote Config

SDK đọc **đúng một** key Firebase Remote Config: `ads_id_config`. Giá trị là toàn bộ JSON ở mục 3.

Ghi đè theo **từng field của từng placement** **1.1.0+**: payload trên Remote Config chỉ cần mang phần thay đổi, field nào không nhắc tới giữ giá trị trong file assets. `ids` có mặt thì thay cả danh sách. Trước 1.1.0 một

placement có trên RC là thay trọn, thiếu `ids` là mất id.

```
// Tắt một chỗ~đặt ad – không cần paste lại ids
{ "placements": { "native_home": { "enable": false } } }

// Bật tầng high-floor cho splash khi đã có inventory
{ "placements": { "inter_splash": { "hf": true } } }

// Đổi id – chỉ placement này, các placement khác giữ nguyên asset
{ "placements": { "banner_home": { "ids": ["ca-app-pub-xxx/new"] } } }
```

Key riêng của app trong `settings` 1.1.0+

Mọi key app đặt trong `settings` đi cùng một lần fetch với id quảng cáo — không phải tự gọi Firebase, và không lệch khoảng fetch/throttle với SDK. Key trên RC ghi đè theo từng key, key thiếu giữ của asset.

```
// Setting SDK tự dùng vẫn có field riêng
val timeout = ads.remoteConfig.splashTimeoutMs ?: 30_000L

// Key của app: getter có kiểu, thiếu hoặc sai kiểu thì ra default
val freeEpisodes = ads.remoteConfig.long("free_episodes_count", 3L)
val rateOnExit = ads.remoteConfig.bool("rate_on_exit", false)

// Kiểm công tắc một chỗ~đặt ad
if (ads.isAdEnabled("native_home")) { /* ... */ }
```

Kiểm tra catalog đang có hiệu lực (asset đã được RC ghi đè): debug build tự in sau mỗi lần nạp RC, hoặc gọi `AdGraph.adUnitCatalog.describe()`. Mỗi placement một dòng kèm nguồn (`asset` , `remote` , `asset+remote`) và id nào đang bị tắt.

Payload hỏng không làm mất ads.

JSON rỗng, null hoặc parse lỗi đều bị bỏ qua và catalog giữ nguyên bản đang dùng. SDK không bao giờ tự đẩy mình về trạng thái rỗng — đó là cách nhanh nhất để mất sạch doanh thu vì một dấu phẩy thừa.

15. Sự kiện và doanh thu quảng cáo 1.1.0+

Mọi sự kiện của mọi format đi qua một listener, chạy trên main thread. Sự kiện quan trọng nhất là

`onAdPaid` : mỗi impression một lần, kèm `AdRevenue` (giá trị AdMob ước tính, đơn vị tiền tệ, độ chính xác) — đúng thứ Adjust / AppsFlyer / Firebase cần để tính ROAS.

```
AdsGraph.addListener(object : AdsListener {
    override fun onAdPaid(placement: String, format: AdFormat, revenue: AdRevenue) {
        adjust.trackAdRevenue("admob_sdk", revenue.value, revenue.currencyCode)
    }
    override fun onAdImpression(placement: String, format: AdFormat) {
        analytics.log("ad_impression_$placement")
    }
    override fun onAdFailedToLoad(placement: String, format: AdFormat, reason: String?) {
        Timber.w("no fill $placement: $reason")
    }
})
```

Sự kiện	Khi nào
<code>onAdLoaded</code> / <code>onAdFailedToLoad</code>	Waterfall của placement kết thúc có / không có ad.
<code>onAdShowed</code> / <code>onAdClosed</code> / <code>onAdFailedToShow</code>	Ad toàn màn (inter, reward, app-open).
<code>onAdImpression</code> / <code>onAdClicked</code>	Mọi format.
<code>onAdRewarded</code>	Người dùng nhận thưởng.
<code>onAdPaid</code>	Impression được định giá. Mọi format.

Firestore `ad_impression` tự động.

App có Firebase Analytics thì SDK tự gửi sự kiện chuẩn `ad_impression` (`ad_platform`, `ad_format`, `ad_unit_name`, `value`, `currency`) cho mỗi impression có giá — đúng schema GA4 dùng để tính ad revenue. Không có Firebase Analytics thì im lặng. Tắt bằng `AdsConfig(logAdImpressionToFirestore = false)` nếu app đã tự gửi.

16. Jetpack Compose 1.1.0+

Module `freshness-ads-compose` bọc ba thứ hay cần trong Compose; interstitial / rewarded / app-open dùng thẳng `AdsGraph.adsManager` trong `LaunchedEffect` hoặc `rememberCoroutineScope` như thường.

```
// Native: preload + shimmer + bind + ẩn khi no-fill, gỡ ad khi rời composition
NativeAd(
    key = "native_detail",
    placement = "native_detail",
    template = { LoadingNativeAdHomeView(it) }, // hoặc LoadingNativeAdTemplateView(it,
R.layout.my_native)
    modifier = Modifier.fillMaxWidth().height(240.dp),
)

// Banner: chiều cao do Google trả, đừng đặt height
BannerAd("banner_home", Modifier.fillMaxWidth())
BannerAd("banner_feed", Modifier.fillMaxWidth(), size = BannerSize.INLINE)

// Pause video khi có ad toàn màn
val adShowing by rememberFullScreenAdShowing()
LaunchedEffect(adShowing) { player.playWhenReady = !adShowing }
```

`NativeAd` không compose gì khi thất bại nên feed dùng nó trong `LazyColumn` sẽ tự khép slot. Mỗi slot một key riêng (cùng placement vẫn phải khác key). Cả hai composable render rỗng trong `@Preview`.

17. ProGuard / R8

Rule của SDK đi kèm trong AAR (`consumer-rules.pro`), tự áp vào app khi minify. Không phải chép gì.

Rule đã bao gồm: model JSON cấu hình, native ad view inflate từ XML, GMA next-gen + mediation adapter, Unity Ads, và constructor Room mà WorkManager gọi bằng reflection.

Nếu release crash ngay khi mở app

với `Failed to create an instance of androidx.work.impl.WorkDatabase`: WorkManager đi kèm GMA và Unity Ads, nó dựng một Room database từ `androidx.startup` — trước cả `Application`. Rule này đã có sẵn trong SDK; chỉ khi bạn tự tắt consumer rules mới cần thêm tay:

```
-keep class * extends androidx.room.RoomDatabase { <init>(); }
```

18. Mediation Unity Ads

Adapter Unity đã nằm trong SDK. Không cần khai gì trong manifest, không cần code init — adapter tự nhận cấu hình từ mediation group phía AdMob lúc chạy.

Phải tạo mediation group trên AdMob TRƯỚC khi phát hành.

Chưa có group thì adapter nhận cấu hình rỗng, nó init bằng một app id giả và *giữ luôn callback init của GMA*. Đo thực tế: `MobileAds.initialize` mất ~30 giây thay vì ~2,2 giây.

Dấu hiệu trong logcat: `SKIP banner ... (SDK not ready)` hoặc `WATERFALL skip ... (SDK not ready)` xuất hiện ngay trước `MobileAds (next-gen) initialized`. Banner và native chờ tới 60 giây nên vẫn lên sau khi init xong, nhưng interstitial và rewarded chỉ chờ 15 giây (có user đang đợi sau spinner) nên vẫn mất.

Sửa ở AdMob, đừng nói thời gian chờ. Triệu chứng nhìn giống hết lỗi code, nhưng nguyên nhân nằm ở cấu hình phía AdMob.

Version của adapter và của Unity SDK phải đi thành cặp: ba số đầu của adapter khớp version SDK nó bọc (adapter `4.18.0.x` bọc `unity-ads 4.18.0`). SDK này ghim cặp `4.18` vì đó là cấu hình đã chạy production cùng GMA next-gen `1.2.1`. Đừng nâng lên bản mới hơn chỉ vì nó mới hơn — adapter mediation phải khớp với GMA next-gen, và chuyện đó chỉ kiểm được bằng một app đang sống.

Từ 1.0.1, SDK cũng ghim `adquality-sdk` ở `9.9.0`: `unity-ads` khai phụ thuộc này bằng dải động (`,10.0.0`), không ghim thì mỗi lần build có thể ra một version khác mà không ai đổi dòng nào.

19. Test và debug

Khi **app host** chạy bản debuggable, SDK **ép test id của Google**, bất kể id thật trong config. Nhờ vậy dev không bao giờ vô tình bấm vào quảng cáo thật (dẫn tới khoá tài khoản AdMob).

1.1.0: "debug" là debug của app host.

Các bản trước đọc `BuildConfig.DEBUG` của chính thư viện — mà AAR trên JitPack là bản release, nên trong app host cờ đó luôn false: debug build của app vẫn chạy id thật, và form consent test EEA không bao giờ hiện. Từ 1.1.0 mọi nhánh debug đọc `ApplicationInfo.FLAG_DEBUGGABLE` của app.

Muốn debug dùng id thật + waterfall thật:

```
AdsConfig(useRealIdsInDebug = true)
```

Bắt buộc phải bật khi cần kiểm hành vi rút tier: với test id thì tier nào cũng fill, và waterfall không bao giờ rút xuống tier sau. Release build bỏ qua hoàn toàn cờ này.

Form consent trên máy test: UMP in hashed id của máy ra logcat ở lần chạy đầu, đưa vào

```
AdsConfig(consentTestDeviceIds = listOf("..."))
```

 . Debug build mặc định ép geography EEA để form hiện; tắt bằng `consentDebugGeographyEea = false` .

Log: SDK dùng Timber và plant cây log khi app debuggable, hoặc khi Remote Config đặt

```
settings.enableLogForTester = true
```

1.1.0+ — tester đọc được log trên bản release mà không cần build riêng. Tag là tên class, lọc bằng grep:

```
adb logcat | grep -E
"InterAdProvider|RewardAdProvider|OpenAdProvider|NativeAdmobManager|AdPoolManager|AdBannerProvider|ConsentProvider|AdsManagerProvider|RemoteConfig"
```

AdMob native ad validator

Trên test device, Google tự hiện một bong bóng cạnh mỗi native ad để chấm bố cục. Đây là cách duy nhất thấy được các vi phạm bố cục native — **bản release không cảnh báo gì, quảng cáo vẫn hiện bình thường**, nên lỗi loại này rất dễ đi thẳng ra production.

Hai lỗi hay gặp và cách sửa:

Validator báo	Nghĩa là	Sửa
<code>MediaView is too small for video</code>	Vùng media dưới 120×120dp	Cho view native một chiều cao cụ thể, dùng để <code>wrap_content</code> với template full-bleed (mục 8)
<code>Advertiser assets outside native ad view</code>	Có asset tràn ra ngoài biên <code>NativeAdView</code>	Thường do container cha bị khoá chiều cao nhỏ hơn nội dung bên trong

Bấm *See issues* trên bong bóng để xem chi tiết. Dùng chính template có sẵn của SDK thì cả 13 template đều được chấm “No implementation issues found”.

20. Những lỗi làm tụt show rate

Show rate = impression ÷ matched request. Con số thấp nghĩa là bạn xin ad về rồi không dùng — và AdMob nhìn thấy điều đó.

Triệu chứng	Nguyên nhân thường gặp	Cách sửa
Một placement chỉ vài phần trăm	Preload ở mọi phiên nhưng chỉ hiển thị ở một nhánh hiếm ai đi vào	Hoặc thôi preload nó, hoặc tìm chỗ hiện nó ở luồng chính
Đúng khoảng 50%	Pool size 2 mà màn chỉ có một slot	Đặt pool size bằng số slot hiển thị cùng lúc
Native cứ vào lại màn là tụt	Hủy ad khi màn chết dù nó chưa từng hiện	Dùng <code>releaseOnDestroy = true</code> — SDK chỉ hủy ad đã ăn impression, fill chưa hiện được giữ lại cho lần sau
App-open thấp kinh niên	Bản chất format: luôn phải ôm sẵn một ad cho lần quay lại có thể không bao giờ tới	Trần lý thuyết là $R / (R+1)$ với R là số lần quay lại mỗi phiên. Đừng đánh đổi impression để kéo con số này lên
Interstitial splash matched mà không show	<code>retryCount</code> lớn làm id chính fill muộn, tới lúc đó user đã rời splash	Đặt <code>retryCount = 1</code> , để waterfall tuần tự xuống tier sau
Rewarded preload nhiều mà ít show	Preload sẵn cho một nút hiếm ai bấm	Dùng <code>loadAndShowRewardAd</code> : bấm rồi mới nạp, màn chờ che khoảng nạp

Nguyên tắc gọn

- Chỉ preload thứ bạn thật sự sẽ hiển thị, ở luồng người ta thật sự đi qua.
- Pool size = số slot nhìn thấy cùng lúc. Không hơn.
- Fill chưa hiện là tiền chưa tiêu — giữ lại, đừng hủy.
- Ad không fill thì thu gọn slot, đừng để shimmer chạy mãi.
- Đừng chặn điều hướng vì một quảng cáo. Ad không về thì cho user đi tiếp.

21. Thay đổi theo phiên bản

Bản	Thay đổi	Cần làm gì khi nâng cấp
1.0.1	Banner chuyển sang <i>large anchored adaptive</i> (API cũ đã deprecated). Ghim <code>adquality-sdk 9.9.0</code> . UMP 3.2.0 → 4.0.0. <code>lifecycle-runtime-ktx</code> chuyển sang <code>api</code> .	Kiểm container banner để <code>wrap_content</code> — banner cao hơn trước.
1.0.2	Banner và native chờ init 60 giây thay vì 15, để không mất trắng khi <code>MobileAds.initialize</code> chạy lâu.	Không cần làm gì.
1.0.3	SDK tự vẽ màn chờ, giữ tối thiểu 500ms. Thêm <code>loadAndShowInterAd</code> / <code>loadAndShowRewardAd</code> . <code>onShow</code> và <code>onReward</code> được đưa về main thread.	App đang tự vẽ màn chờ riêng thì đặt <code>showDefaultLoadingUi = false</code> để khỏi có hai spinner chồng nhau.
1.0.4	Sửa lỗi màn chờ không tắt khi quảng cáo hiện. Màn chờ đổi sang nền trắng, spinner xanh.	Đang dùng 1.0.3 thì nên nâng — lỗi cũ làm spinner hiện lại sau khi đóng quảng cáo.
1.0.5	Giữ sàn 120dp cho <code>MediaView</code> ở hai template full-bleed.	Đang dùng <code>HomeView</code> / <code>DisplayView</code> với <code>wrap_content</code> thì card sẽ cao hơn — kiểm lại bố cục.
1.0.6	Sửa <code>CtaTopView</code> tràn asset ra ngoài native ad view và chữ body không đọc được. Thêm lớp phủ tối cho <code>FullscreenView</code> .	Đang dùng <code>CtaTopView</code> thì card cao thêm 5dp và chữ body đổi màu.
1.1.0	Compose: module <code>freshness-ads-compose</code> với <code>NativeAd</code>, <code>BannerAd</code>, <code>rememberFullScreenAdShowing</code>. Native: <code>LoadingNativeAdTemplateView</code> / <code>NativeAdTemplateView</code> bind layout XML của app theo id quy ước. Rewarded interstitial: format <code>rewardedInter</code>, <code>loadAndShowRewardedInterAd</code>. Analytics: <code>AdsGraph.addListener(AdsListener)</code> với <code>onAdPaid(AdRevenue)</code> cho mọi format, tự gửi <code>ad_impression</code> lên Firebase. Full-screen: ad đã nạp hết hạn sau 1 giờ; <code>showInterAdOutcome</code> / <code>showRewardAdOutcome</code> trả <code>AdShowOutcome</code>; <code>isFullScreenAdShowing</code> gộp ba format; <code>openAdExcludedActivities</code>. Native: <code>NativeAdOptions</code> (video tắt tiếng mặc định, tỉ lệ media, <code>AdChoices</code>) ghi đề được per placement; nạp lại slot sau click và sau video. Banner: <code>BannerSize</code> <code>ANCHORED</code> / <code>INLINE</code> / <code>MREC</code>. <code>settings.enableLogForTester</code> bật log trên release. Remote Config <code>ads_id_config</code> ghi đề theo từng field (chỉ gửi phần thay đổi); thêm <code>hf</code>, <code>ids</code> dạng chuỗi thuần, <code>settings</code> key tùy ý với <code>remoteConfig.long/bool/string</code>, <code>adUnitCatalog.describe()</code>. Nhánh debug đọc cò debuggable của app host (trước đây không bao giờ bật trong app host). Firebase và Crashlytics thành tùy chọn. Remote Config: dùng giá trị đã activate khi fetch lỗi/throttle, khoảng fetch 1 giờ ở release. Interstitial có guard chống hai lần show chồng nhau; callback GMA hop về main trước khi sửa state. <code>loadAndShowInterAd</code> kiểm frequency cap trước khi nạp.	Native video giờ mặc định tắt tiếng — muốn giữ tiếng thì <code>NativeAdOptions(videoMuted = false)</code>. App đang gọi <code>Timber.plant</code> riêng không bị ảnh hưởng. Payload RC cũ vẫn chạy nguyên (field thiếu giờ giữ của asset thay vì rỗng — chỉ khác khi RC cố ý gửi placement thiếu <code>ids</code>). Đang set <code>AdsGraph.adUnitCatalog.useRealIds</code> thì đổi sang <code>AdsConfig(useRealIdsInDebug = true)</code>. Đang gọi thẳng class <code>*Provider</code> thì đi qua <code>AdsGraph</code>. Splash nên thêm <code>isShowLoading = false</code>. Debug build của app giờ thật sự dùng test id — muốn id thật thì bật cò trên.

Thêm `AdsConfig.isPremium`, `useRealIdsInDebug`,
`consentTestDeviceIds`; overload ngắn
`showInterAd("key")`,
`loadAndShowRewardAd("key"); resetInterTimer`
lên interface. `isCollapsible` deprecated. Các provider
chuyển `internal`.